Data Cleaning - Introduction

Jan Nimczik

Load packages

```r
library(dplyr)
library(Hmisc)
library(VIM)
```

Data is the new. . .

Artificial Intelligence, however you want to define it, that's everything. There will be more changes in the next five to seven years than we've seen in the last 30. It will impact every business. Data is the new gold. It's the new oil. It's the new plastics. (Marc Cuban)

Important Skills

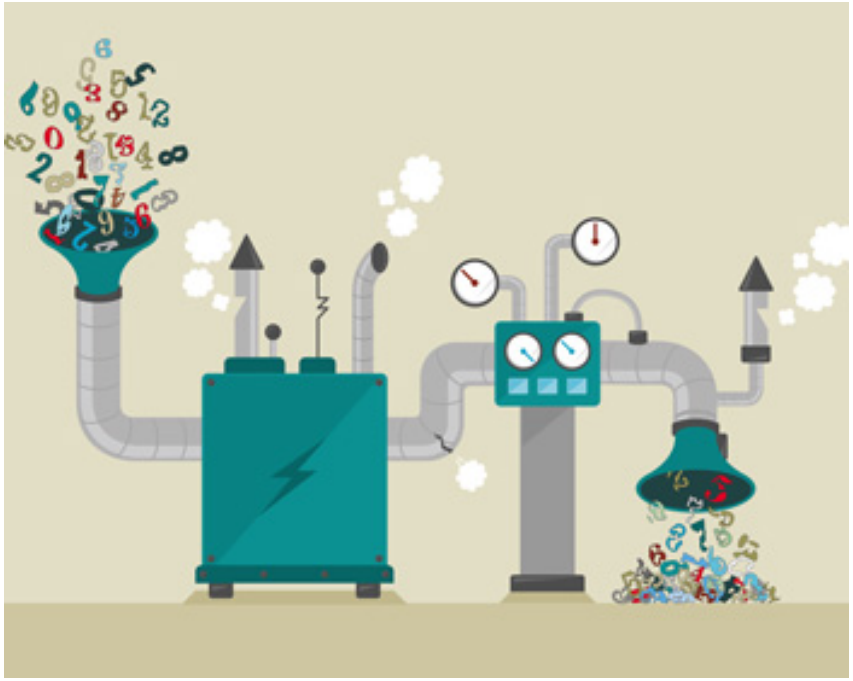
The ability to take data – to be able to understand it, to process it, to extract value from it, to visualize it, to communicate it's going to be a hugely important skill in the next decades, not only at the professional level but even at the educational level for elementary school kids. Because now we really do have essentially free and ubiquitous data. (Hal Varian)

Steps in any Data Project (e.g. your thesis)

1. ask an interesting question
2. get the data
3. (pre-)process the data
4. explore the data
5. model the data
6. communicate and visualize the results

Today: Step 3 - Data (Pre-)Processing

- Data has errors
- "Garbarge in, garbage out"



Step 3 - Data (Pre-)Processing

- Data munging/Data wrangling/Data cleaning
  - art of preparing the data for analysis in order to get an informative, manageable data set.
  - important to get technically correct and consistent data
  - often the most time consuming part of an analysis
- Other important aspects: Data storage and Data management in order to be able to access data quickly and reliably during subsequent analysis

Step 3 - Data (Pre-)Processing

To do list:

- create copy of original data (never overwrite!)
- work in systematic and reproducible way (write script!)
- perform "sniff test" to detect artifacts (really look at the data!)
- use summary statistics and visualization to understand the data
- unify units
- deal with missing values
- deal with outliers

Types of data

Please go to menti.com

Plan for today

- we will use an example data set and go through the to do list
- we will use R (of course you can use other programs, but. . .)
- you will get the code after class

Plan for today

- Part 1: read in and understand the data
- Part 2: clean the data

Some general advice about programming

- You will find nearly everything on google and hence also on ChatGPT (try "length of a list in R")
- Old version: A programmer is someone who can turn stack overflow snippets into running code
- New version: A programmer is someone who can use ChatGPT, Claude et al. as an effective assistant to produce running code
- Use auto-completion provided by R-studio
- Sign up at GitHub co-pilot (it's free for you!)
- Learn keyboard shortcuts
- Make your variable names meaningful

Part 1: Read in Data

Our example data is a small subset of a large data set that collects all advertisements on the online platform `immobilienscout24`, the largest platform of housing ads in Germany (with a market share of more than 50%). The data is stored as a `csv` file named `apartment_rents.csv`.

https://www.immobilienscout24.de/Suche/de/berlin/berlin/wohnung-mieten?numberofrooms=1.0-&enteredFrom=one_step_searchlink

Read in Data

First hurdle:

- often data comes in text format and is not nicely preprocessed
- character encodings
- different variable types
  - numerical
  - factor variables
  - strings
  - dates

Data Format

Storing data as `csv` file is an elegant solution because this can be read by basically all programs. More difficult to work with proprietary software.

Character encodings

Have you seen something like this, e.g., when trying to work with German Umlauts?

J	K	L	M	N
CT_MA	gemeinde_1970	gemeinde_2015	kreis_1970	regierungsbezirk_1970
10.000	Heilbronn	Heilbronn	Stadtkreis H	NordwÃ¼rttemberg
10.000	Stuttgart	Stuttgart	Stadtkreis S	NordwÃ¼rttemberg
10.000	Ulm	Ulm	Stadtkreis U	NordwÃ¼rttemberg
10.000	Aalen, GroÃŸe Kreis	Aalen	Aalen	NordwÃ¼rttemberg
10.000	AbtsgmÃ¼nd	AbtsgmÃ¼nd	Aalen	NordwÃ¼rttemberg
10.000	Adelmannsfelden	Adelmannsfelder	Aalen	NordwÃ¼rttemberg
10.000	Aufhausen	Bopfingen	Aalen	NordwÃ¼rttemberg
10.000	Baldern (Bopfingen	Bopfingen	Aalen	NordwÃ¼rttemberg
10.000	Benzenzimmern	Kirchheim am Ri	Aalen	NordwÃ¼rttemberg
10.000	Baldern (Bopfingen	Bopfingen	Aalen	NordwÃ¼rttemberg
10.000	Dalkingen	Rainau	Aalen	NordwÃ¼rttemberg
10.000	Dewangen	Aalen	Aalen	NordwÃ¼rttemberg
10.000	Dirgenheim	Kirchheim am Ri	Aalen	NordwÃ¼rttemberg
10.000	Dorfmerkingen	Neresheim	Aalen	NordwÃ¼rttemberg
10.000	Ebnat	Aalen	Aalen	NordwÃ¼rttemberg
10.000	Elchingen auf dem	Neresheim	Aalen	NordwÃ¼rttemberg
10.000	Ellenberg	Ellenberg	Aalen	NordwÃ¼rttemberg
10.000	Ellwangen (Jagst), S	Ellwangen (Jagst	Aalen	NordwÃ¼rttemberg
10.000	Essingen	Essingen	Aalen	NordwÃ¼rttemberg
10.000	Fachsenfeld	Aalen	Aalen	NordwÃ¼rttemberg

Character encodings

UTF-8 is by far the most common encoding for the World Wide Web, accounting for 97% of all web pages, and up to 100% for some languages, as of 2021.

```
options(encoding = "utf-8")
```

Let's load our data

```r
rents <- read.csv("files/apartment_rents.csv")
```

Data types

Use `str()` function to get an overview on the structure of the data.

```r
str(rents)
```

```
## 'data.frame':    200000 obs. of  19 variables:
##  $ ID                  : int  88980903 108120398 54969195 107940043 53769777
##  $ exclusive_rent      : num  402 247 630 346 220 ...
##  $ living_area         : num  53 36.7 70 55 49 ...
##  $ number_of_rooms     : num  2 1 2 2 2 3 3 3 2 4 ...
##  $ additional_costs    : num  85 121 170 130 127 180 130 NA 121 105 ...
##  $ year_of_construction: int  1953 1974 1968 1967 1996 1970 1958 1976 1915 1
##  $ balcony             : chr  "Yes" "Yes" "No" "Yes" ...
##  $ lift                : chr  "No" "Yes" "No" "Yes" ...
##  $ fitted_kitchen      : chr  "No" "No" "No" "No" ...
##  $ guest_toilet        : chr  "No" "No" "No" "No" ...
##  $ equipment           : chr  NA NA "Normal" NA ...
##  $ click_customer      : chr  "Variable nicht mehr in Einagbemaske vorhanden
##  $ ags2019             : int  14612000 11000000 6412000 5913000 14713000 531
##  $ flat_category       : chr  "apartment" NA "loft apartment" "apartment" ..
##  $ city_or_municipality: chr  "Dresden" "Berlin" "Frankfurt am Main" "Dortmu
##  $ administrative_unit : chr  "Stadt" "Stadt" "Stadt" "Stadt" ...
##  $ federal_state       : chr  "Sachsen" "Berlin" "Hessen" "Nordrhein-Westfal
##  $ offer_year          : int  2016 2018 2010 2018 2016 2012 2017 2019 2014 2
##  $ offer_month         : int  6 11 4 10 4 5 9 11 5 8 ...
```

Data types

- numerical
- integer
- factor
- character
- dates and times

Summary Statistics

```r
summary(rents)
```

```
##        ID            exclusive_rent    living_area     number_of_rooms
##  Min.   : 25573904   Min.   : 144.0   Min.   : 19.10   Min.   :0.000
##  1st Qu.: 53280641   1st Qu.: 360.0   1st Qu.: 53.00   1st Qu.:2.000
##  Median : 68246996   Median : 500.0   Median : 66.00   Median :2.000
##  Mean   : 71350475   Mean   : 614.6   Mean   : 70.14   Mean   :2.483
##  3rd Qu.: 88040307   3rd Qu.: 757.0   3rd Qu.: 83.00   3rd Qu.:3.000
##  Max.   :115101898   Max.   :2400.0   Max.   :165.00   Max.   :8.000
##                                                        NA's   :35
##  additional_costs year_of_construction   balcony              lift
##  Min.   :   0.0   Min.   :1500         Length:200000      Length:200000
##  1st Qu.:  96.0   1st Qu.:1931         Class :character   Class :character
##  Median : 131.0   Median :1967         Mode  :character   Mode  :character
##  Mean   : 146.1   Mean   :1962
##  3rd Qu.: 180.0   3rd Qu.:1994
##  Max.   :1460.0   Max.   :2020
##  NA's   :12092    NA's   :73383
##  fitted_kitchen     guest_toilet        equipment          click_customer
##  Length:200000      Length:200000      Length:200000      Length:200000
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##
##
##
##
```

Select Variables

```
rents <- select(rents,
            exclusive_rent, additional_costs, living_area,
            number_of_rooms, balcony, lift, fitted_kitchen,
            guest_toilet, city_or_municipality, offer_year,
            year_of_construction, equipment, flat_category)
```

Coercion

```r
dat <- read.csv(file = "files/unnamed.txt",
        header = FALSE, col.names = c("age","height"),
        stringsAsFactors=FALSE)
dat$height <- as.numeric(dat$height)
```

```
## Warning: NAs introduced by coercion
```

```r
# this is called coercion. So if we want to coerce
# some input into a numeric we type:
as.numeric(c("7", "7*", "7.0", "7,0"))
```

```
## Warning: NAs introduced by coercion
```

```
## [1]  7 NA  7 NA
```

Time instances (dates)

```r
current_time <- Sys.time()
class(current_time)
```

```
## [1] "POSIXct" "POSIXt"
```

```r
current_time
```

```
## [1] "2025-04-30 14:27:47 CEST"
```

lubridate package

- contains functions that facilitate conversion of text to `POSIXct` dates

```
library(lubridate)
dates <- c("13/01/2020", "13 Jan 19",
           "It happened on 13 01 '20")
dmy(dates)
```

Additional info on times and dates

- not uncommon to indicate years with two numbers, leaving out the indication of century. In R , 00-68 are interpreted as 2000-2068 and 69-99 as 1969-1999.

```r
dmy("01 01 68")
```

```
## [1] "2068-01-01"
```

```r
dmy("01 01 69")
```

```
## [1] "1969-01-01"
```

```r
#some abbreviations can not be recognized
dmy("15 Febr. 2013")
```

```
## Warning: All formats failed to parse. No formats found.
```

```
## [1] NA
```

Additional info on times and dates (continued)

```r
dates <- c("15-9-2009", "16-07-2008", "17 12-2007",
           "29-02-2011")
as.POSIXct(dates, format = "%d-%m-%Y")
```

```
## [1] "2009-09-15 CEST" "2008-07-16 CEST" NA
```

```r
mybirth <- dmy("17 May 1984")
format(mybirth, format = "I was born on %B %d, %Y")
```

```
## [1] "I was born on May 17, 1984"
```

Character type

```r
library(stringr)

#trim white space
str_trim(" hello world ")
str_trim(" hello world ", side = "left")
str_trim(" hello world ", side = "right")
```

Character type (continued)

```r
#pad strings with certain characters to a specific width
str_pad(112, width = 6, side = "left", pad = 0)
```

```
## [1] "000112"
```

```r
#convert to upper or lower case
toupper("Hello world")
```

```
## [1] "HELLO WORLD"
```

```r
tolower("Hello World")
```

```
## [1] "hello world"
```

Matching strings

Imagine you want to combine data from two different sources.

- sometimes this is easy because you have the same ID in both data sets (e.g. stock symbols like MSFT)
- sometimes this is a mess because you have to match on text
  - many ways how people write down the same things
  - use package `stringr`

Matching strings - example

- finding substring in strings

```r
countries <- c("germany", "GER", "U.S.A.", "USA")
grepl("g", countries)
```

```
## [1]  TRUE FALSE FALSE FALSE
```

```r
grep("g", countries)
```

```
## [1] 1
```

```r
grepl("g", countries, ignore.case = TRUE)
```

```
## [1]  TRUE  TRUE FALSE FALSE
```

```r
grepl("g", tolower(countries))
```

```
## [1]  TRUE  TRUE FALSE FALSE
```

Matching strings (continued)

- search for strings that start with g

```
grepl("^g", countries, ignore.case = TRUE)
```

```
## [1]  TRUE  TRUE FALSE FALSE
```

- matching close strings (approximate matching)

```
codes <- c("GER", "USA")
D <- adist(countries, codes, ignore.case = TRUE)
colnames(D) <- codes
rownames(D) <- countries
D
```

```
##          GER USA
## germany    4   6
## GER        0   3
## U.S.A.     6   3
## USA        3   0
```

Matching strings (continued)

- choose the one with the closest distance
- use apply to apply which.min to each row of D

```
i <- apply(D, 1, which.min)
data.frame(rawtext = countries, coded = codes[i])
```

```
##   rawtext coded
## 1 germany   GER
## 2     GER   GER
## 3  U.S.A.   USA
## 4     USA   USA
```

- for more check out the package `stringdist`

Part 2: Data Cleaning

- Distinguishing errors from artifacts.
- Data compatibility / unification.
- Imputation of missing values.
- Outlier detection.

Errors vs. Artifacts

- Data errors represent information that is fundamentally lost in acquisition.
- Artifacts are systematic problems arising from processing done to data.

Tip: The key to detecting artifacts is the sniff test, examining the product closely enough to detect anomalities.

Getting rid of obvious inconsistencies

- e.g., negative age, pregnant man etc.

```
print(head(arrange(rents, number_of_rooms)))
```

```
##   exclusive_rent additional_costs living_area number_of_rooms balcony lift
## 1         385.00           100.00       75.00               0      No   No
## 2         230.00           120.00       28.00               0     Yes <NA>
## 3         420.00            80.00       44.10               0    <NA> <NA>
## 4         324.00               NA       56.50               0     Yes <NA>
## 5         484.51            65.95       83.25               0      No   No
## 6         451.78           140.56       97.34               0      No   No
##   fitted_kitchen guest_toilet city_or_municipality offer_year
## 1             No           No           Düsseldorf       2009
## 2           <NA>         <NA>    Frankfurt am Main       2007
## 3           <NA>         <NA>    Frankfurt am Main       2007
## 4             No         <NA>               Bremen       2007
## 5             No         <NA>              Hamburg       2007
## 6             No         <NA>              Hamburg       2007
##   year_of_construction equipment flat_category
## 1                   NA    Normal     apartment
## 2                   NA      <NA>          <NA>
## 3                   NA      <NA>          <NA>
## 4                   NA      <NA>     apartment
## 5                 1981      <NA>          <NA>
## 6                 1973      <NA>          <NA>
```

Apartment with zero rooms?

Probably a data error.

```
rents <- filter(rents, number_of_rooms >= 1)

print(head(arrange(rents, number_of_rooms)))
```

```
##   exclusive_rent additional_costs living_area number_of_rooms balcony lift
## 1         247.30              121       36.66               1     Yes  Yes
## 2         400.00               90       45.00               1     Yes <NA>
## 3         475.00              184       38.00               1      No   No
## 4         202.00               60       27.50               1     Yes   No
## 5         323.04               68       38.00               1      No   No
## 6         537.00              110       48.82               1      No  Yes
##   fitted_kitchen guest_toilet city_or_municipality offer_year
## 1             No           No               Berlin       2018
## 2            Yes         <NA>           Düsseldorf       2008
## 3            Yes           No                 Köln       2016
## 4             No           No             Dortmund       2013
## 5            Yes           No              Hamburg       2010
## 6            Yes           No              Hamburg       2012
##   year_of_construction equipment  flat_category
## 1                 1974      <NA>           <NA>
## 2                 1999      <NA> loft apartment
## 3                 1960      <NA>      apartment
## 4                 1958    Normal loft apartment
## 5                 1965    Normal      apartment
## 6                 1970      <NA>      apartment
```

Data Compatibility

Data needs to be carefully massaged to make “apple to apple” comparisons

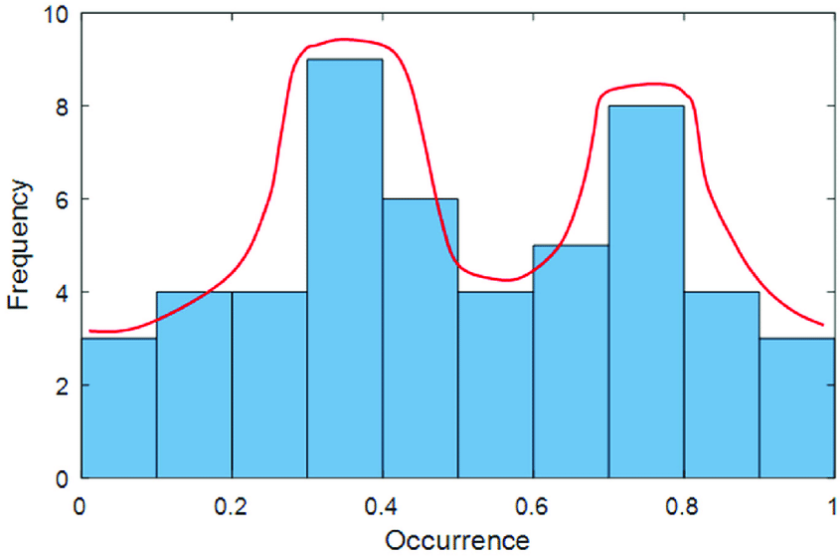
Numeric representations

- The Ariane 5 rocket exploded in 1996 due to a bad 64-bit float to 16-bit integer conversion.
- Measurements should generally be decimal numbers
- Counts should be integers.
- Fractional quantities should be decimal, not (q,r) like (pounds,oz) or (feet,inches).

Unit conversions

Look at the distribution of every variable:

- Bimodal distributions can indicate trouble



Name unification

- Use simple transformations to unify names, like lower case, removing middle names, etc.

Time/date unification

- Aligning temporal events from different datasets/systems can be problematic.
- Use Coordinated Universal Time (UTC), a modern standard subsuming GMT.
- Financial time series are tricky because of weekends and holidays: how do you correlate stock prices and temperatures?

Financial unification

- Currency conversion uses exchange rates.
- Correct stock prices for splits and dividends.
- Use returns / percentage change instead of absolute price changes.
- The time value of money needs correction for inflation for fair long-term comparisons.
- Why do stock/oil prices correlate over 30 years?

Simple corrections

```r
#data contains obvious inconsitencies
(international.rents <- read.csv("files/international_rents.csv",
                  stringsAsFactors = FALSE))
```

```
##   X   city   rent currency
## 1 1    NYC   1400      USD
## 2 2 Berlin    800     Euro
## 3 3 Zurich   1040      CHF
## 4 4 Moskow  31000      RUB
## 5 5  Tokyo 150000      YEN
```

Simple corrections (continued)

```r
(international.rents.cor <- mutate(international.rents,
      rent = ifelse(currency == "USD",rent*0.82,rent),
      rent = ifelse(currency == "CHF", rent*0.92, rent),
      rent = ifelse(currency == "RUB", rent*0.011, rent),
      rent = ifelse(currency == "YEN", rent*0.0079, rent),
      currency = "Euro"))
```

```
##   X   city   rent currency
## 1 1    NYC 1148.0     Euro
## 2 2 Berlin  800.0     Euro
## 3 3 Zurich  956.8     Euro
## 4 4 Moskow  341.0     Euro
## 5 5  Tokyo 1185.0     Euro
```

Missing Values

An important aspect of data cleaning is properly representing missing data:

- What is the year of death of a living person?
- What about a field left blank or filled with an obviously outlandish value?
- The frequency of events too rare to see?

Tip: Setting such values to zero is generally wrong.

Missing Values

```r
summary(rents)
```

```
##  exclusive_rent   additional_costs  living_area     number_of_rooms
##  Min.   : 144.0   Min.   :   0.0   Min.   : 19.10   Min.   :1.000
##  1st Qu.: 360.0   1st Qu.:  96.0   1st Qu.: 53.00   1st Qu.:2.000
##  Median : 500.0   Median : 131.0   Median : 66.00   Median :2.000
##  Mean   : 614.6   Mean   : 146.1   Mean   : 70.14   Mean   :2.483
##  3rd Qu.: 757.0   3rd Qu.: 180.0   3rd Qu.: 83.00   3rd Qu.:3.000
##  Max.   :2400.0   Max.   :1460.0   Max.   :165.00   Max.   :8.000
##                   NA's   :12085
##    balcony              lift           fitted_kitchen     guest_toilet
##  Length:199957      Length:199957      Length:199957      Length:199957
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##
##
##
##
##  city_or_municipality   offer_year   year_of_construction  equipment
##  Length:199957        Min.   :2007   Min.   :1500         Length:199957
##  Class :character     1st Qu.:2010   1st Qu.:1931         Class :character
##  Mode  :character     Median :2013   Median :1967         Mode  :character
##                       Mean   :2013   Mean   :1962
##                       3rd Qu.:2016   3rd Qu.:1994
##                       Max.   :2019   Max.   :2020
##                                      NA's   :73354
```

Imputation

- With enough training data, one might drop all records with missing values.
- Often, however, it is better to estimate or impute missing values instead of leaving them blank.

Modes of imputation:

- Heuristic based imputation - A good guess for death year is birth+80.
- Mean value imputation - leaves mean same.
- Random value imputation - repeatedly selecting random values permits statistical evaluation of the impact of imputation.
- Imputation by interpolation - using linear regression to predict missing values works well if few fields are missing per record.
- Imputation by nearest neighbor - requires a distance function to identify the most similar records.

Tip: There is no single best imputation method that works in all cases.
Important difference to excel: R does not impute data where it shouldn't, Excel does!!!

Missing values

```r
rent <- c(350, 300, 420, NA)
mean(rent)
```

```
## [1] NA
```

```r
mean(rent, na.rm = TRUE)
```

```
## [1] 356.6667
```

na.omit

- `na.omit` creates a data.frame where incomplete rows have been deleted
- row.names of the removed records are stored in `na.action`

```
rents.complete <- na.omit(rents)
dim(rents.complete)
```

```
## [1] 42716    13
```

We loose a lot of information if we simply delete cases with missing values.

Heuristic-based imputation

- impute value "No" in case that balcony, lift, kitchen, or guest toilet is missing

```
rents$balcony[is.na(rents$balcony)] <- "No"
rents$lift[is.na(rents$lift)] <- "No"
rents$fitted_kitchen[is.na(rents$fitted_kitchen)] <- "No"
rents$guest_toilet[is.na(rents$guest_toilet)] <- "No"

rents.complete <- na.omit(rents)
dim(rents.complete)
```

```
## [1] 47138    13
```

Imputing the mean or other centrality measures

```r
rent
```

```
## [1] 350 300 420  NA
```

```r
rent <- impute(rent, mean)
rent
```

```
##          1          2          3          4
##  350.0000  300.0000  420.0000 356.6667*
```

```r
is.imputed(rent)
```

```
## [1] FALSE FALSE FALSE  TRUE
```

Ratio imputation

- `x = Ry` where `x` is missing and `y` is a covariate
- imputation by linear regression: use information on covariates to predict the values of missings

```
model <- lm(number_of_rooms ~ living_area +
              exclusive_rent + balcony,
            data = rents)
# automatically omits rows with missing values
```

Ratio imputation (continued)

```r
summary(model)
```

```
##
## Call:
## lm(formula = number_of_rooms ~ living_area + exclusive_rent +
##     balcony, data = rents)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -4.2391 -0.3553 -0.0179  0.3490  4.9803
##
## Coefficients:
##                  Estimate Std. Error t value Pr(>|t|)
## (Intercept)     5.016e-01  3.774e-03 132.888  < 2e-16 ***
## living_area     3.156e-02  6.929e-05 455.399  < 2e-16 ***
## exclusive_rent -3.953e-04  4.882e-06 -80.984  < 2e-16 ***
## balconyYes      1.663e-02  2.620e-03   6.348 2.19e-10 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.5458 on 199953 degrees of freedom
## Multiple R-squared:  0.6201, Adjusted R-squared:  0.6201
## F-statistic: 1.088e+05 on 3 and 199953 DF,  p-value: < 2.2e-16
```

Ratio imputation (continued)

```r
I <- is.na(rents$number_of_rooms)
rents$number_of_rooms[I] <- predict(model, newdata = rents[I, ])

summary(is.na(rents$number_of_rooms))
```

```
##    Mode   FALSE
## logical  199957
```

Random imputation

```r
additional_costs <- rents$additional_costs[20:35]
additional_costs
```

```
##  [1]  87 135 150  95  70 119 260  91 230  88 160 106 200 175  NA 170
```

```r
(additional_costs <- impute(additional_costs, "random"))
```

```
##    1    2    3    4    5    6    7    8    9   10   11   12   13   14   15
##   87  135  150   95   70  119  260   91  230   88  160  106  200  175 175*
```

k-nearest-neighbor imputation

```r
library(VIM)
```

k-nearest-neighbor imputation (continued)

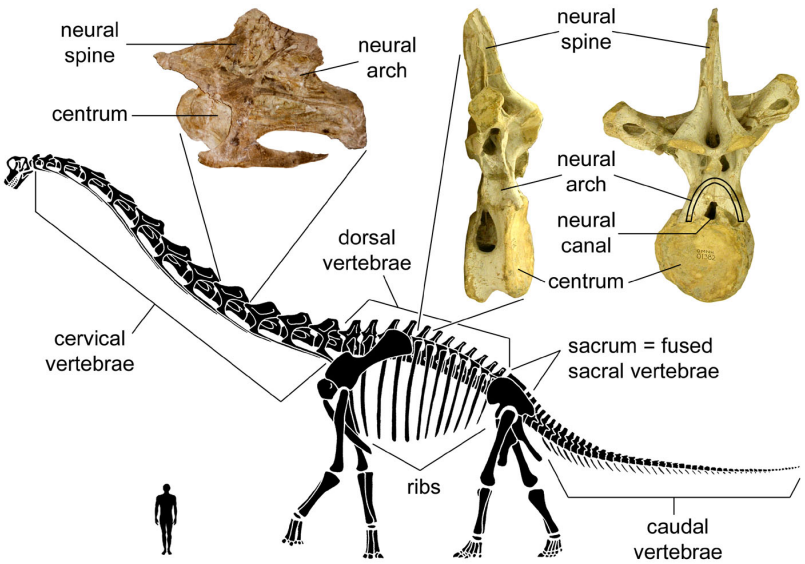
```r
# consider a smaller dataframe because KNN is time-intensive
apartment.rent.knn <- rents %>% select(exclusive_rent, living_area, number_of_r
n <- nrow(apartment.rent.knn)
# provide some empty values (10 in each column, randomly)
for (i in 1:ncol(apartment.rent.knn)) {
  apartment.rent.knn[sample(1:n, 10, replace = FALSE), i] <- NA
}
apartment.rent2 <- kNN(apartment.rent.knn)
head(apartment.rent2)
```

```
##   exclusive_rent living_area number_of_rooms balcony     equipment
## 1            265       46.73               2     Yes        Normal
## 2           1100       87.00               3     Yes Sophisticated
## 3           1472       95.00               3      No Sophisticated
## 4            540       74.17               3     Yes        Normal
## 5            750       85.00               2      No        Normal
## 6            673       49.87               2      No        Normal
##   exclusive_rent_imp living_area_imp number_of_rooms_imp balcony_imp
## 1              FALSE           FALSE               FALSE       FALSE
## 2              FALSE           FALSE               FALSE       FALSE
## 3              FALSE           FALSE               FALSE       FALSE
## 4              FALSE           FALSE               FALSE       FALSE
## 5              FALSE           FALSE               FALSE       FALSE
## 6              FALSE           FALSE               FALSE       FALSE
##   equipment_imp
## 1         FALSE
## 2         FALSE
```

Outliers

The largest reported dinosaur vertebra is 50% larger than all others: presumably a data error.

- Look critically at the maximum and minimum values for all variables.
- Normally distributed data should not have large outliers, k sigma from the mean.



Tip: Fix why you have an outlier. Don't just delete.

Outliers

- Visually, it is easy to detect outliers, but only in low dimensional spaces.
- It can be thought of as an unsupervised learning problem, like clustering.
- Points which are far from their cluster center are good candidates for outliers.

Tip: Deleting outliers prior to fitting can yield better models, e.g. if these points correspond to measurement error. Deleting outliers prior to fitting can yield worse models, e.g. if you are simply deleting points which are not explained by your simple model.

boxplot.stats to list outliers

```r
x <- rents$exclusive_rent[1:50]
boxplot.stats(x)$out
```

```
## [1] 1400.00 1395.00 1290.00 1700.00 1618.61
```

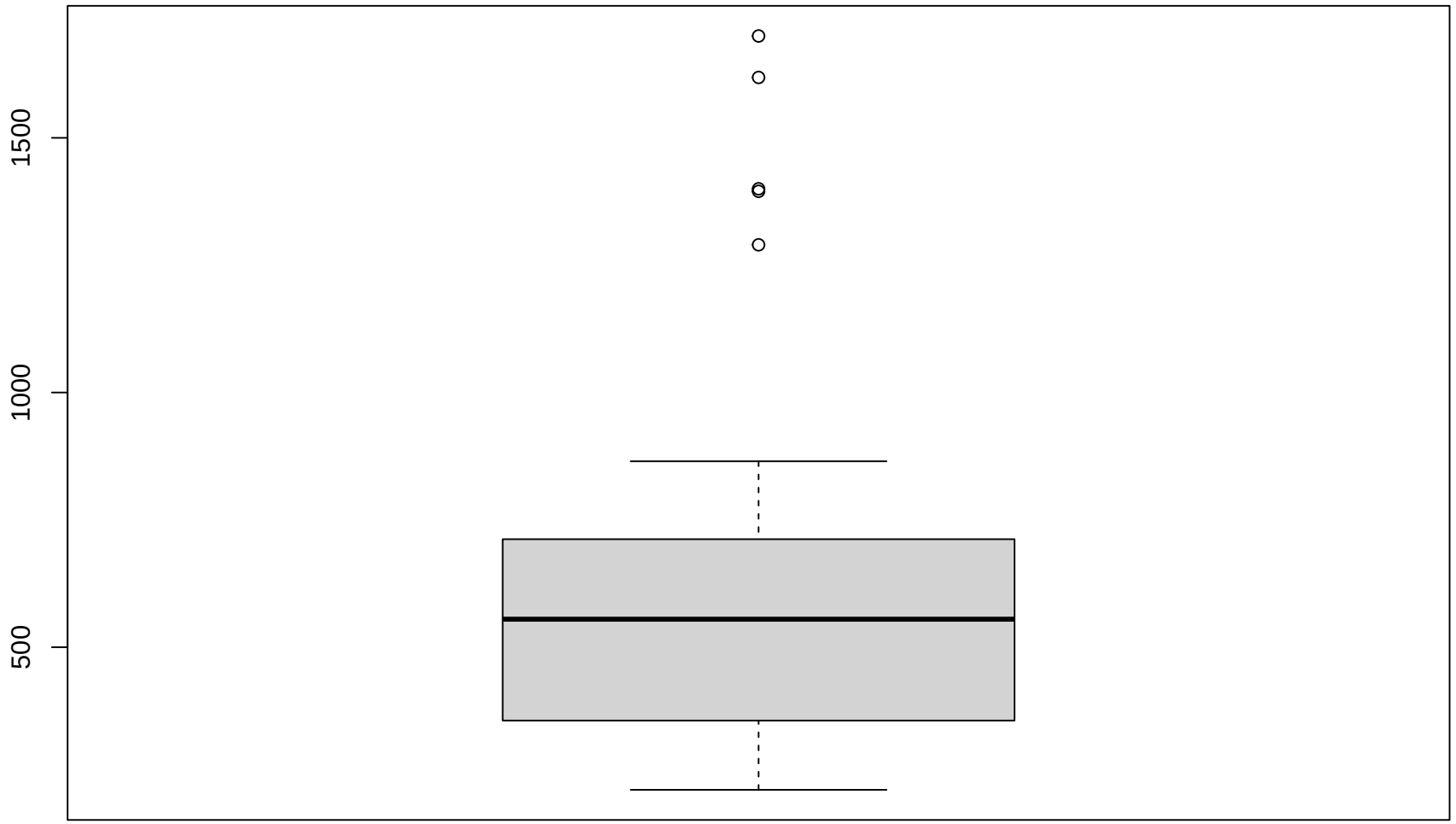
- increase outlier detection limit:

```r
boxplot.stats(x, coef = 2)$out
```

```
## [1] 1700.00 1618.61
```

boxplot.stats to plot outliers

```
boxplot(x, coef = 2)
```



Trimming

Trimming means simply removing the outliers. This can for instance be done when calculating a mean:

```
mean(rents$exclusive_rent)
```

```
## [1] 614.5821
```

```
mean(rents$exclusive_rent, trim = 0.01)
```

```
## [1] 605.0077
```

```
cut_point_top <- quantile(rents$exclusive_rent, 0.975)
cut_point_bottom <- quantile(rents$exclusive_rent, 0.025)

rents$exclusive_rent_trim <- rents$exclusive_rent
rents$exclusive_rent_trim[rents$exclusive_rent_trim > cut_point_top] <- NA
rents$exclusive_rent_trim[rents$exclusive_rent_trim < cut_point_bottom] <- NA

head(rents[[2]])
```

```
## [1]  85 121 170 130 127 180
```

Winsorizing

Winsorizing means replacing the values of outliers by the cut value, i.e., the maximum (or minimum) value of the variable for the non-outliers.

```
rents$rent_wins <- rents$exclusive_rent
rents$rent_wins[rents$rent_wins > cut_point_top] <- cut_point_top
rents$rent_wins[rents$rent_wins < cut_point_bottom] <- cut_point_bottom
```